

xv6 Scheduling: MLFQ, FIFO and Round Robin

CS3.301 Operating Systems and Networks, Mini Project 1, Part 2

Chandrani Saha

chandrani.saha@research.iiit.ac.in

Overview

This report covers the scheduler half of the mini project. I implemented a four level feedback queue (MLFQ) in xv6-riscv, added a non-preemptive FIFO scheduler so there was something to compare against, and kept the stock round robin (RR) scheduler as the default. A new system call, `waitx`, reports turnaround, waiting, response and running time for each child, and a user program called `schedulertest` generates a fixed workload with four different CPU burst lengths.

Section 1 walks through every change the specification asks about. Section 2 checks the MLFQ against a tick by tick trace. Section 3 compares the three schedulers on two workloads. All numbers in this report come from my own runs on a single CPU (`CPUS=1`), measured in timer ticks, and every plot was produced by `plot_mlfq.py`, which is submitted with the code.

1 Implementation Summary

1.1 Makefile and the SCHEDULER macro

I added a `SCHEDULER` variable to `xv6/Makefile`. Setting it to `MLFQ` or `FIFO` adds `-DMLFQ` or `-DFIFO` to `CFLAGS`, while leaving it unset (or setting it to `RR`) compiles the original round robin path with no scheduler macro at all. Any other value stops the build with an error, because a typo such as `SCHEDULER=mlfq` used to build round robin without complaint. I also added `TRACE=1`, which defines `MLFQTRACE` and makes an MLFQ kernel print one trace line per tick, and I added `schedulertest` to `UPROGS`. Since make does not notice when `CFLAGS` change, a `make clean` is needed between policies, otherwise old object files are reused.

```
make clean && make qemu                                # round robin (default)
make clean && make qemu SCHEDULER=MLFQ
make clean && make qemu SCHEDULER=FIFO
make clean && make qemu SCHEDULER=MLFQ TRACE=1 CPUS=1
```

1.2 Changes to struct proc

I added eight fields to `struct proc` in `kernel/proc.h`, all protected by `p->lock` just like `state`.

Field	Type	Meaning
<code>queue</code>	<code>int</code>	MLFQ priority, 0 is the highest and 3 the lowest
<code>slice_used</code>	<code>int</code>	ticks already used from the current queue's slice
<code>enter_seq</code>	<code>uint64</code>	ticket from a global counter; a smaller ticket is nearer the head of its queue
<code>ctime</code>	<code>int</code>	tick at which the process was created
<code>etime</code>	<code>int</code>	tick at which it exited (0 while alive)
<code>rtime</code>	<code>int</code>	ticks spent RUNNING
<code>wtime</code>	<code>int</code>	ticks spent RUNNABLE, which is time in the ready queue
<code>first_run</code>	<code>int</code>	tick of its first dispatch, or <code>-1</code> before that

There are no linked lists for the queues. A process's position is simply its queue number plus its ticket, and the scheduler treats "lowest queue, then lowest ticket" as the head. Pushing a process to the tail of a queue just means giving it a fresh ticket. I chose this because the fields live under the `p->lock` the kernel already takes while scanning the process table, so there is no extra queue lock to order against `p->lock` and `wait_lock`, which is exactly where schedulers like this tend to deadlock. The timing fields are compiled into every build because the comparison needs the same numbers from FIFO and RR. `NQUEUE` (4) and `BOOST_INTERVAL` (48) live in `kernel/param.h`.

1.3 Changes to `allocproc()`

`allocproc()` now places every new process in queue 0 with an empty slice, records `ctime = ticks`, zeroes `etime`, `rtime` and `wtime`, and sets `first_run` to `-1`. `freeproc()` clears the same fields so a recycled slot starts clean. The ticket itself is taken in `kfork()` and `userinit()` at the moment the process is marked RUNNABLE, through a small helper called `requeue()`. That way the child really joins the tail of queue 0 when it becomes ready, and not earlier, while `kfork()` is still copying memory. `kexit()` records `etime`. No explicit removal is needed on exit, since a ZOMBIE is never RUNNABLE and the scheduler never picks it.

1.4 Queue selection and preemption

Under MLFQ, `scheduler()` scans the whole process table and picks the RUNNABLE process in the lowest numbered queue, breaking ties with the smallest ticket. That gives strict priority between queues and FIFO order inside each one. Each lock is released during the scan, so after choosing, the scheduler locks the process again and checks that its state, queue and ticket have not changed; if another CPU or a boost got there first, it simply scans again.

For preemption, `usertrap()` and `kerneltrap()` in `kernel/trap.c` call `yield()` on a timer interrupt only when `mlfq_tick()` says so. That happens in two cases: the current process has used its whole slice, or `mlfq_higher_waiting()` finds a RUNNABLE process in a strictly higher queue. So a process running in queue 2 is preempted at the next tick boundary as soon as something arrives in queue 0, which is the example in the specification. A process preempted this way keeps its queue and its partly used slice, since it did nothing to deserve a demotion.

1.5 Time slice handling

Slices are {1, 4, 8, 16} ticks for queues 0 to 3. Once per tick, `clockintr()` on CPU 0 calls `update_time()`, which adds a tick to `rtime` and `slice_used` for every RUNNING process and to `wtime` for every RUNNABLE one. I placed this call before `ticks++` and `wakeup(&ticks)`, so a process that slept through the tick is counted as asleep rather than waiting. On the next timer interrupt of the CPU running the process, `mlfq_tick()` compares `slice_used` to the slice for its queue. When the slice is used up the process drops one queue (or stays in queue 3), gets a

fresh ticket at the tail and starts a new slice, which also makes queue 3 behave as round robin. On several CPUs the slice can run out between two of that CPU's own timer checks, so `sleep()` repeats the check before a process goes to sleep; without it, sleeping at that moment would dodge the demotion.

1.6 Voluntary yield handling

A process that sleeps on I/O, `pause()`, a pipe or `wait` leaves the ready set simply by not being `RUNNABLE`, so going to sleep needed no change. When `wakeup()` or `kkill()` makes it runnable again, it calls `requeue()`, which hands out a fresh ticket so the process joins the tail of the same queue it was in. Its priority is unchanged, as the rule requires. I also reset `slice_used` on wakeup. The specification does not say what happens to a partly used slice, and resetting it is the simpler reading, with one known cost: a process that always sleeps just before its slice ends keeps its priority. The trace in Section 2 shows exactly this for the burst 150 child.

1.7 Priority boosting

Every 48 ticks, `clockintr()` calls `mlfq_boost()`, which moves every live process (running, runnable, sleeping or still being set up) to queue 0 and clears its slice. `UNUSED` and `ZOMBIE` slots are skipped because they are not part of any queue. I left the tickets alone, so processes keep their relative order when they land in queue 0. The boost is what bounds starvation, both for CPU bound processes sitting in queue 3 and for any process kept down by a stream of new arrivals in queue 0.

1.8 procdump changes

Under MLFQ, Ctrl+P now prints each process's queue, its slice progress, its ticket, its running and waiting totals, and how many ticks remain until the next boost:

```
7 runble schedulertest  q2 slice 2/8 seq 212  run 34 wait 75  boost in 33
```

I used this constantly while developing to watch processes fall through the queues and jump back at a boost. The countdown to the next boost carries the same information as “ticks since the last boost” (48 minus the countdown). RR and FIFO builds keep the original output.

1.9 Supporting pieces

waitx. System call 23, `int waitx(int *turnaround, int *waiting, int *response, int *running)`, behaves like `wait` but also copies out four statistics, which are read before `freeproc()` clears them: `turnaround = etime - ctime`, `waiting = wtime`, `response = first_run - ctime` and `running = rtime`.

FIFO. Built with `SCHEDULER=FIFO`, the scheduler picks the `RUNNABLE` process with the smallest ticket, and the timer interrupt never calls `yield()`. Tickets are handed out whenever a process becomes runnable, so this is a proper ready queue: whoever became ready first runs next and keeps the CPU until it sleeps or exits.

Round robin. The policy is untouched: the timer yields every tick, so the quantum is one tick. The only additions on this path are the timing bookkeeping (`first_run` and `update_time()`), which never affects a scheduling decision.

schedulertest. `schedulertest [nproc] [rounds]` forks up to 16 children. Each child repeatedly burns the CPU for a burst and then calls `pause(1)` to give it up voluntarily. Bursts cycle through 1, 40, 150 and 600 units of work, which under QEMU is roughly a fraction of a tick, 2, 7 and 30 ticks. Short bursters get proportionally more rounds (120, 48, 17 and 5 with `rounds = 5`) so they do not all finish long before the long ones. The parent reaps every child with `waitx` and prints each child's numbers and the averages.

2 MLFQ Analysis

2.1 How the timeline was produced

I built the kernel with `SCHEDULER=MLFQ TRACE=1 CPUS=1`, ran `schedulertest 4 5` and saved the console output. For every tick, CPU 0 prints one line per RUNNING or RUNNABLE process in the form `MLFQTRACE <tick> <pid> <queue> <running>`, and `plot_mlfq.py` turns those lines into Figure 1. Sleeping processes are not sampled, so they appear as gaps. The small vertical offset between pids is only there so overlapping lines stay visible. Tracing slows the kernel, so this run is used only for the timeline and never for timing numbers.

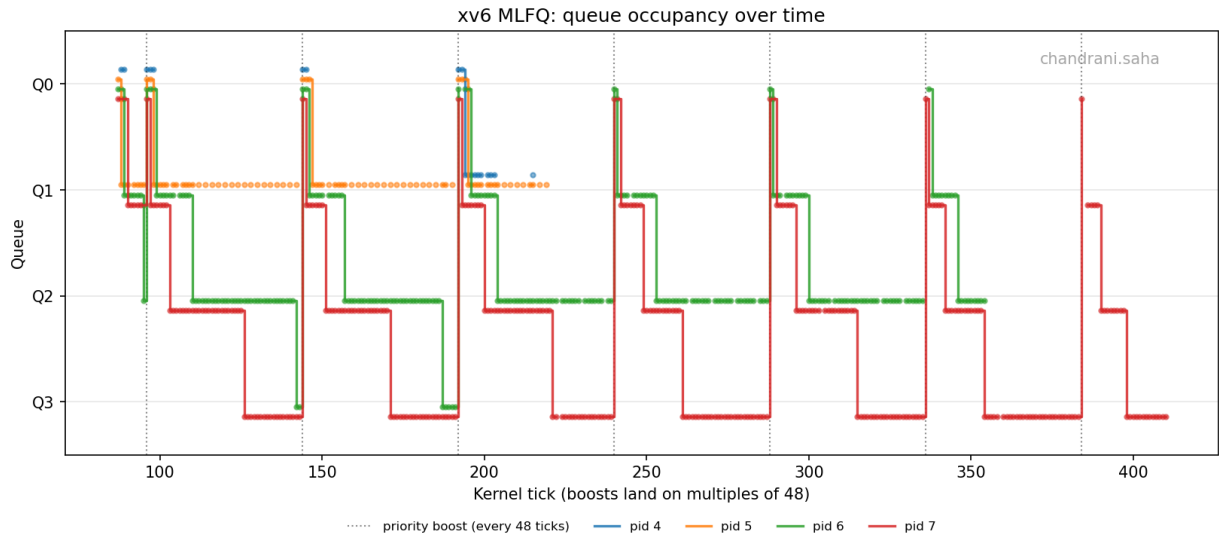


Figure 1: Queue of each `schedulertest` child over time (`schedulertest 4 5`, one CPU). pid 4 has a burst of 1 unit, pid 5 of 40, pid 6 of 150 and pid 7 of 600. Dotted lines mark the priority boost every 48 ticks.

2.2 What the trace shows

The workload ran from tick 87 to tick 410. I checked the trace line by line with a script, and the counts it gives match the kernel's own `waitx` numbers for that run exactly (running 1, 51, 120 and 149 ticks, waiting 18, 35, 132 and 171 ticks for pids 4 to 7).

pid (burst)	Q0 run/wait	Q1 run/wait	Q2 run/wait	Q3 run/wait	Total run/wait	Sleeps seen
4 (1)	1 / 8	0 / 10	0 / 0	0 / 0	1 / 18	5
5 (40)	4 / 5	47 / 30	0 / 0	0 / 0	51 / 35	47
6 (150)	7 / 6	35 / 28	78 / 91	0 / 7	120 / 132	16
7 (600)	8 / 4	29 / 18	60 / 54	52 / 95	149 / 171	4

Table 1: Ticks spent running and waiting in each queue, counted from the trace. pid 4 is mostly invisible because most of its bursts start and end between two samples.

Transition	Slice (ticks)	Demotions seen	Running ticks before each one
Q0 to Q1	1	20	exactly 1 in all 20
Q1 to Q2	4	14	exactly 4 in all 14
Q2 to Q3	8	9	exactly 8 in all 9
Q3 to the tail of Q3	16	1	exactly 16

Table 2: Every demotion in the trace, counting slice use from the last wakeup (a wakeup starts a fresh slice).

A few more checks came out clean. On none of the 324 ticks was a runnable process waiting in a better queue than the one running, and no tick went by with the CPU idle while something was runnable. At each of the seven boosts from tick 96 to tick 384, every process present was in queue 0. Higher priority wakeups preempted a lower priority runner 44 times, for example pid 5 waking in queue 1 and displacing pid 7 in queue 2 at ticks 112 to 124.

2.3 Interpretation

pid 7 (burst 600) is the clearly CPU bound process. It slept only 4 times in 324 ticks, each of its descents from queue 0 used the full 1, 4 and 8 tick slices, and it spent 95 of its 171 waiting ticks in queue 3. pid 5 (burst 40) is I/O bound: it slept 47 times, almost always after about a tick of work, so it never left queue 1 (47 of its 51 running ticks) and kept jumping ahead of the longer jobs. pid 6 (burst 150) sits in between. It reached queue 3 twice, but after tick 221 it slept every 6 or 7 running ticks, and since a wakeup starts a fresh 8 tick slice it stayed in queue 2 and never actually ran in queue 3. The boost is what kept pid 7 from starving behind the others: it got 37 of its 149 running ticks (about 25%) in queues 0 and 1 right after boosts, and then sank back to queue 3 within 14 to 30 ticks each time.

3 Comparison Results

3.1 Method

I built each scheduler with `CPUS=1`, booted a fresh kernel and ran `schedulertest 4 5` and `schedulertest 8 3`, without tracing. Both workloads use the same four burst lengths; 8 3 simply has two children of each length and fewer rounds. Times are in ticks, and lower is better for every metric.

3.2 Averages

Scheduler	schedulertest 4 5			schedulertest 8 3		
	Turnaround	Waiting	Response	Turnaround	Waiting	Response
FIFO	265.50	148.00	2.00	308.63	236.25	17.75
RR	274.00	152.00	0.75	327.00	253.88	2.25
MLFQ	197.00	75.25	0.75	239.13	166.88	2.25

Table 3: Average turnaround, waiting and response time. The best value in each column is in bold.

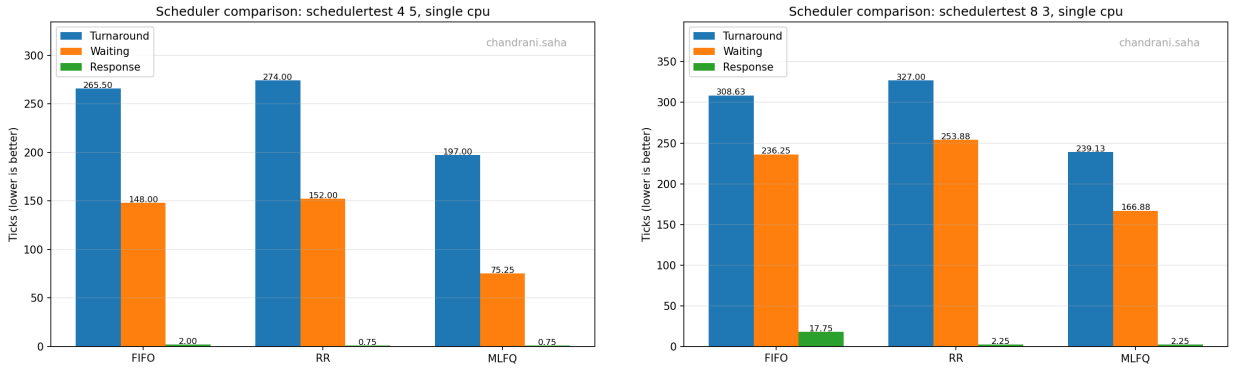


Figure 2: The averages from Table 3 as bar charts, for `schedulertest 4 5` (left) and `schedulertest 8 3` (right).

Comparison	4 5			8 3		
	Turnaround	Waiting	Response	Turnaround	Waiting	Response
MLFQ against RR	-28.1%	-50.5%	equal	-26.9%	-34.3%	equal
MLFQ against FIFO	-25.8%	-49.2%	-62.5%	-22.5%	-29.4%	-87.3%
FIFO against RR	-3.1%	-2.6%	2.7×	-5.6%	-6.9%	7.9×

Table 4: Relative differences between the averages. A negative percentage means the first scheduler is lower (better) by that much; 2.7× means 2.7 times higher.

3.3 Who wins and who pays

Averages hide the most interesting part, which is how each policy treats short and long bursts. Table 5 and Figure 3 split the results by burst length.

Workload	Burst	MLFQ (T / W / R)	RR (T / W / R)	FIFO (T / W / R)
4 5	1	126 / 6 / 0	273 / 153 / 0	339 / 219 / 0
	40	126 / 29 / 0	233 / 135 / 0	311 / 202 / 0
	150	236 / 116 / 1	302 / 159 / 1	250 / 131 / 1
	600	300 / 150 / 2	288 / 161 / 2	162 / 40 / 7
8 3	1	82.0 / 10.0 / 1.5	328.0 / 256.0 / 1.5	389.0 / 314.5 / 15.5
	40	175.5 / 103.0 / 1.5	274.5 / 217.0 / 1.5	353.5 / 281.5 / 15.5
	150	353.5 / 284.5 / 2.5	361.0 / 275.0 / 2.5	294.0 / 225.5 / 17.0
	600	345.5 / 270.0 / 3.5	344.5 / 267.5 / 3.5	198.0 / 123.5 / 23.0

Table 5: Turnaround (T), waiting (W) and response (R) by burst length. For 8 3 each value is the mean of the two children with that burst.

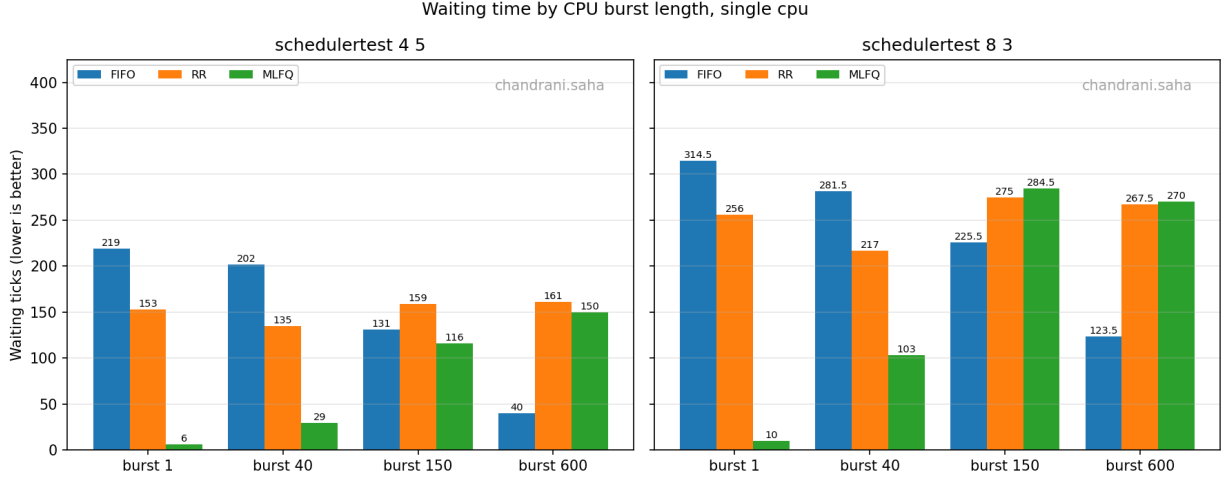


Figure 3: Waiting time for each burst length under each scheduler.

3.4 Discussion

MLFQ gave the lowest average turnaround and waiting time in both workloads, 197.00 and 75.25 ticks for 4 5 against 274.00 and 152.00 for RR. Almost all of that gain comes from the short bursters: a burst 1 child never uses up its one tick slice in queue 0, so it runs almost as soon as it wakes and waited just 6 ticks over 120 rounds, while under RR it waited 153 and under FIFO 219. RR had the worst waiting time in both workloads because its quantum is a single tick, so every wakeup has to queue behind a full tick of every other runnable job, and the burst 40 child, which needs a little over a tick, gets cut off and has to go round the whole queue again to finish. That is why RR’s waiting time depends so much on the quantum: a larger quantum would let those jobs finish in one go but make short jobs sit behind longer turns, and once the quantum is longer than every burst, RR becomes FIFO. FIFO shows the opposite trade-off, because nothing ever takes the CPU away: the burst 600 child finished first (turnaround 162 in 4 5) while the short jobs got stuck behind whole 24 to 30 tick bursts, the classic convoy effect. For response time, MLFQ and RR tied exactly, since each new child starts in queue 0 and waits at most one tick for every earlier child. FIFO looked fine in 4 5 (2.00) only because the long job happened to be forked last, but in 8 3 the second group of children had to wait for a whole 24 tick burst before their first dispatch, which pushed the average to 17.75, almost eight times RR’s. MLFQ’s gains are not free: the long jobs pay for them, and the burst 600 children waited 270 ticks in 8 3 compared with 123.5 under FIFO, mostly getting the CPU only after a boost.

3.5 Limitations

- Time is sampled once per tick on CPU 0, so a burst shorter than a tick is usually charged nothing. That is why the burst 1 child shows `running` 0, and a process sharing a tick can be charged the whole tick.
- Response time only counts the very first dispatch. After that, a burst 1 child under FIFO waits about 4 ticks per round in 8 3, which the response column never shows.
- Every configuration was run once, on a single CPU. The total CPU work agrees within 2% across schedulers for 8 3 (352 to 359 ticks), but FIFO’s 4 5 run recorded about 6% less than the other two (280 against 297 and 298), most likely because QEMU ran a little faster during that run. The gap between RR and FIFO (3 to 7%) is small enough that it should not be read too strongly; the MLFQ results are far outside that margin.

- Tracing is too slow to time, so the traced run (average turnaround 216.75) is used only for the timeline.

3.6 Correctness checks

Beyond the numbers, I ran xv6's own `usertests -q`. It passes under MLFQ on one and on three CPUs, and under RR. FIFO hangs at the `preempt` test, as any non-preemptive scheduler must, because that test needs a spinning child to be taken off the CPU. Run on its own right after boot, `usertests reparent` can fail under MLFQ: `init` is demoted during boot and a steady stream of new queue 0 processes keeps it off the CPU until the first boost while zombies fill the process table. That is strict priority starving a low priority process, which the 48 tick boost bounds, and the test passes inside the full suite.